

Automated Penetration Testing with LLM Agents and Classical Planning

Lingzhi Wang*, Xinyi Shi*, Ziyu Li*, Yi Jiang[†], Shiyu Tan[†], Yuhao Jiang*, Junjie Cheng[†], Wenyan Chen[†],
Xiangmin Shen[‡], Zhenyuan LI[†], Yan Chen*

*Northwestern University, [†]Zhejiang University, [‡]Hofstra University

Abstract—While penetration testing plays a vital role in cybersecurity, achieving fully automated, hands-off-the-keyboard execution remains a significant research challenge. In this paper, we introduce the “Planner-Executor-Perceptor (PEP)” design paradigm and use it to systematically review existing work and identify the key challenges in this area. We also evaluate existing penetration testing systems, with a particular focus on the use of Large Language Model (LLM) agents for this task. The results show that the out-of-the-box Claude Code and Sonnet 4.5 exhibit superior penetration capabilities observed to date, substantially outperforming all prior systems. However, a detailed analysis of their testing processes reveals specific strengths and limitations; notably, LLM agents struggle with maintaining coherent long-horizon plans, performing complex reasoning, and effectively utilizing specialized tools. These limitations significantly constrain its overall capability, efficiency, and stability. To address these limitations, we propose CHECKMATE, a framework that integrates enhanced classical planning with LLM agents, providing an external, structured “brain” that mitigates the inherent weaknesses of LLM agents. Our evaluation shows that CHECKMATE outperforms the state-of-the-art system (Claude Code) in penetration capability, improving benchmark success rates by over 20%. In addition, it delivers substantially greater stability, cutting both time and monetary costs by more than 50%.

Index Terms—Cyberattacks, Penetration Testing, LLM, Classical Planning

I. INTRODUCTION

Penetration testing, hereafter used interchangeably with “pentesting,” has become a vital component of cybersecurity. It enables organizations to proactively identify and mitigate vulnerabilities before exploited by adversaries. The U.S. Cybersecurity and Infrastructure Security Agency (CISA) highlights pentesting as a core service for protecting critical infrastructure [1]. MarketsandMarkets [2] forecasts that the global pentesting market was valued at USD 1.7 billion in 2024, and is projected to grow to USD 3.9 billion by 2029 [3]. The PenTesting-as-a-Service (PTaaS) segment alone is expected to expand from USD 118 million in 2024 to USD 301 million by 2029. This significant and rising demand has prompted the adoption of artificial intelligence (AI) to meet the need for more efficient and scalable penetration testing solutions. A recent research [4] suggests that by 2028, AI-powered security testing tools will outnumber human pentesters in mainstream security operations, signaling a transformative shift in the field.

The high reliability of skilled human pentesters has made fully automated penetration testing a long-standing challenge in cybersecurity research. However, existing approaches

still require nontrivial human intervention and have yet to achieve truly “hands-off-the-keyboard” automation [5]. As a result, users are often forced to act as supervisors, executors, prompters, and decision-makers, limiting the practical utility of these systems. Moreover, human intervention hinders rigorous and unbiased evaluation, as it is difficult to standardize the extent of human involvement during evaluation, leading to inconsistent assessments across studies. In real-world use, different human knowledge, experience, and skill levels make it harder for the system to achieve its expected effectiveness.

Multiple challenges have prevented truly “hands-off-the-keyboard” automation in pentesting. Earlier work based on formal models [6], [7] is unable to interpret heterogeneous, unstructured information or generate specific commands. Therefore, they either operate on overly simplified scenarios or produce abstract attack plans without concrete commands or instructions. Recent advances in LLMs enable heterogeneous information processing and detailed instruction generation, yet human involvement remains necessary in LLM-based pentesting. First, LLM hallucinations often lead to incomplete, incorrect, or even fabricated commands, requiring human supervision to identify and correct them before execution [5]. Second, limited context memory and logical reasoning capabilities [8]–[11] make LLMs ineffective at performing long-term, multi-step pentesting tasks [5]. They often become stuck in repetitive and unproductive iterations and require human input and guidance.

Recently, LLMs have been widely applied to complex tasks such as software development [12]–[14], code auditing [15], [16], and data engineering [17], prompting a focus on *LLM Agents*. These systems are characterized by their ability to automatically plan for complex tasks, execute code and commands, and iteratively refine them based on execution results, significantly reducing the need for human guidance and intervention. In this paper, we first try to answer the question: *Can LLM agents conduct pentesting independently?* To answer this, we evaluated and compared their penetration capabilities against prior work. Our results show that the out-of-the-box Claude Code [18] and Sonnet 4.5 can autonomously complete pentesting tasks and achieve state-of-the-art (SOTA) capability, substantially outperforming all prior work in this field. To understand the strong performance of Claude Code, we further analyze its testing process and find that it demonstrates superior code refinement and subtask management capabilities. However, we also identify several limitations in Claude Code

and other LLM agents in pentesting, including difficulties in maintaining coherent long-horizon plans, performing complex reasoning, and leveraging specialized tools. Such issues lead to lower success rates, efficiency, and unstable performance.

In this paper, we address these limitations by integrating classical planning techniques with LLM agents. Our approach is motivated by several key insights. First, classical planning represents plans as directed acyclic graphs (DAGs) [19], which provide an explicit and logically structured view of the overall task. This structure enables the agent to maintain a coherent long-horizon plan and mitigates common failure modes, such as erratic attack-path selection, incomplete or repetitive execution, and forgetting previously gathered information. Second, classical planning explicitly encodes causal relationships through preconditions and effects. This allows us to generate long reasoning chains without relying on the LLM itself, thereby ensuring their correctness while maintaining efficient and stable reasoning. Moreover, this explicit causal structure enables us to incorporate customized domain knowledge and reasoning patterns that go beyond the LLM’s internal knowledge. Last but not least, the use of modular actions defined in classical planning allows us to easily integrate uncommon or highly specialized pentesting tools, reducing reliance on the LLM’s internal knowledge for command generation. These predefined actions also let us specify critical execution details in advance, eliminating the need for repeated LLM iterations to refine simple commands on the fly. This design further improves accuracy, efficiency, and overall system stability.

However, traditional classical planning is limited to fully observable and deterministic environments, making it unsuitable for pentesting scenarios. To address this gap, we propose *Classical Planning+*, the first LLM-augmented classical planning framework capable of dynamic updates. Classical planning+ preserves the core principles of classical planning while leveraging LLMs to update action effects and state information on the fly, thereby removing the requirement to fully specify all domain knowledge beforehand. As a result, it extends classical planning to partially observable and non-deterministic domains, significantly broadening its applicability to real-world pentesting tasks.

In this paper, we aim to enhance LLM-based pentesting with classical planning+. We decompose a pentesting system into three core components: a planner, an executor, and a perceptor. Using this design paradigm, we review the design choices in existing work and introduce our system, CHECKMATE. In our system, classical planning+ serves as the planner: it infers the set of feasible actions given the current attack state and selects the most appropriate next step. LLM agents act as the executor that carries out these actions. Their execution outputs are then converted by an LLM into predicates compatible with classical planning+, enabling the planner to update the attack state and determine subsequent steps. Our experiments on the Vulhub [20] dataset demonstrate that CHECKMATE significantly outperforms existing systems, including Claude Code, in penetration capability, efficiency, and stability.

Our main contributions are summarized as follows.

- 1) **Proposed a Unified Design Paradigm for Pentesting.** We propose a design paradigm for automated pentesting systems, which consists of three fundamental components: a Planner, an Executor, and a Perceptor (PEP). We use this paradigm to review and categorize existing work. This paradigm helps us understand the distinctions among existing works and provides guidance for the future development and improvement of such systems.
- 2) **Performed the Largest Evaluation on Existing Systems.** We systematically evaluate existing pentesting systems on the Vulhub dataset. We find that out-of-the-box Claude Code, powered by Sonnet 4.5, achieves the strongest performance with the least human intervention, showing a substantial improvement over all prior work in this area. We further analyzed the penetration workflow of Claude Code and identified three major limitations.
- 3) **Proposed Classical Planning+ and Developed CHECKMATE.** To address these limitations, we propose classical planning+, the first dynamically updating classical planning scheme powered by LLMs, which extends conventional classical planning to partially observable and non-deterministic tasks. Based on it, we develop CHECKMATE, which integrates classical planning and LLM agents for pentesting. Our extensive evaluation demonstrates that CHECKMATE substantially outperforms prior systems, improving success rates on Vulhub benchmark by over 20% and cutting both time and monetary costs by more than 50%.

II. THE PEP PARADIGM AND RELATED WORK

A. The PEP Designing Paradigm for Pentesting Systems

With the growing volume of work in automated pentesting, we consolidate a design paradigm that decomposes an automated pentesting system into three cooperating components: a Planner, an Executor, and a Perceptor (PEP). This decomposition provides a clear way to dissect systems and a framework for future research, allowing each component to be independently analyzed, improved, and benchmarked. Using the PEP paradigm, we taxonomized prior work and, for each component, reviewed representative solutions, remaining challenges, and open problems. The prior work taxonomy is summarized in Table I.

1) *Planner*: The planner aims to answer: (1) what actions are feasible now; (2) among those feasible actions and their following attack paths, which one has the highest value and should be given priority for execution. Earlier work used formal methods as the planner. For example, some work [6], [21]–[24] presents pentesting as a partially observable Markov decision process (POMDP) to model the uncertainty and incomplete information, where the planner must choose between information-gathering (e.g., scans) and offensive (e.g., exploit) actions, update a Bayesian-style belief state from observations, and enumerate feasible actions under the current belief state. Follow-up research has extended POMDP to account for factors such as defender responses [21] or observation noise [23]–[25]. However, these systems suffer computational

blow-up as the problem scales [6]. Moreover, their estimated probability models (e.g., scan success rates or exploitability probabilities) are difficult to obtain in the real world. Another line of work [7], [26]–[29] formulates planning as a path-search problem. CHAINREACTOR [26], as a representative work, casts privilege-escalation attacks as a classical planning task. However, those approaches treat pentesting as a static and deterministic problem, and thus do not address how to plan under those real-world uncertainties and incomplete information.

LLMs can be naturally used as a planner by supplying pentesting information as context and requesting the next action. However, it suffers from some limitations, such as hallucinations, short-term memory, and limited context windows. Therefore, existing work has designed various mechanisms to help LLMs plan. For example, some systems maintain structured textual representations of the plan, including the Penetration Tree (PTT) and its variants in PENTESTGPT [30], AUTOPENTESTER [31], and PENHEAL [32], as well as the Situation Summaries in AUTOATTACKER [33]. VulnBot [34] leverages LLMs to convert the planning problem to a Penetration Task Graph (PTG). PENTESTAGENT [35] uses LLMs together with CVE/service keywords to drive a two-stage planning. It leverages LLMs to interpret the reconnaissance results and extract CVEs or service names, which are used to search for exploits online. Multi-agent systems like CAI [4] rely on LLMs to coordinate available agents or tools. In these frameworks, the LLM has function descriptions and pentesting information as context and determines which agent or tool to invoke at each step. Some systems also maintain a to-do list to guide the agent’s progress and prevent LLMs from being distracted.

Challenges and open questions: Existing pentesting planners can be broadly classified into two categories based on whether they rely on LLMs. The first category comprises traditional planning methods such as POMDPs and classical planning. These approaches offer a clear logical structure. However, they struggle to scale to real-world pentesting environments, where both the state space and action space are large, dynamic, and highly complex. The second category relies on LLMs to perform planning. These approaches do not require formal definitions of actions or states, which greatly simplifies system design. However, the resulting plans often suffer from logical inconsistency and poor long-term coherence. Moreover, the black-box nature of LLMs makes the planning difficult to control, interpret, or systematically improve. Therefore, there are two research directions. The first one is automatically extracting structured knowledge to enable formal planning algorithms in complex penetration scenarios. The second one is enhancing LLM-based planning so that the generated plans exhibit stronger logical consistency and coherence.

2) *Executor*: The executor is responsible for (1) translating the planning results into concrete, executable commands and (2) executing those commands on real systems. Systems without LLMs can only provide commands within a very narrow scope (e.g., a small set of Metasploit exploitations). Some

of them (e.g., CHAINREACTOR) require human operators to execute the generated attack plans [26]. LLMs enable fine-grained generation of commands and code, leading to LLM-driven executors in pentesting, such as the executor of PENTESTGPT. However, these executors cannot interact with target environments, so the generated commands still depend on human operators to execute. With the rise of LLM-based tool calling, systems like PENTESTAGENT, AUTOPENTESTER, CAI are able to synthesize, execute, and iterate commands, significantly reducing human intervention. Retrieval-augmented generation (RAG) has also been widely adopted in executor. PENTESTAGENT, AUTOPENTESTER, VULNBOT, and PENHEAL, for example, combine retrieved code snippets, articles, and previous actions with RAG pipelines to improve the quality of LLM-generated commands.

Challenges and open questions: A major challenge for executors is simulating human-like behaviors and interactions. Many attack vectors only appear through GUIs or interactive workflows where text-only commands and tools are less effective, especially in web penetration scenarios. Mimicking human behaviors (e.g., mouse and keyboard actions) triggers such attack vectors and helps evade defenses. Although the Computer-User Interaction Simulation Agent (CUA) has been proposed to mimic human behaviors in interface operations [36], [37], no existing work has yet applied it for pentesting. Another challenge is effectively leveraging specialized tools that may lie outside an LLMs’ training data.

3) *Perceptor*: The perceptor is responsible for converting heterogeneous, unstructured data, such as tool outputs and error messages, into representations that the planner can use. When planning relies solely on LLMs, this unstructured data can be provided directly to the LLM as context, so a dedicated perceptor is unnecessary. For planners that depend on structured intermediate representations (e.g., PTTs or to-do lists), the perceptor uses an LLM to translate heterogeneous data into the data structures, such as a PTT branch or an item in the to-do list. For a classical planner, unstructured information is mapped to symbolic predicates, either through manually crafted rules or by an LLM.

Challenges and open questions: Existing work focuses primarily on textual data, while visual information is also important in pentesting. For example, analysts may need to infer a web application’s functionality from its user interface or extract data embedded within images (e.g., reading CAPTCHA). Developing future perceptrs, therefore, requires addressing the challenges of robust visual understanding. Although an increasing number of LLMs and multimodal models now offer image-analysis capabilities, to the best of our knowledge, no prior work has leveraged visual artifacts effectively in the context of pentesting.

B. Classical Planning

Classical planners operate on state representations explicitly defined by predicates. Every action includes clearly defined preconditions and effects, which allow the planner to know exactly which actions are applicable at any given state. This

TABLE I: Taxonomy of automated penetration testing systems based on the PEP design paradigm.

System	Planner	Executor	Perceptor
CHAINREACTOR	Classical Planning	Predefined Actions + Human Operators	Rules + LLM (PDDL predicates)
PENTESTGPT	LLM + Penetration Tree	LLM + Human Operators	LLM
AutoPT	LLM + Finite State Machine	LLM + Agents	LLM
PENTESTAGENT	LLM + CVE-Exploit Mapping	LLM + RAG (code snippets) + Agents	LLM
AutoAttacker	LLM + Situation Summary	LLM + RAG (previous tasks) + Agents	LLM
VULNBOT	LLM + Penetration Task Graph	LLM + RAG (previous tasks) + Agents	LLM
PENHEAL	LLM + Penetration Tree	LLM + RAG (previous commands) + Agents	LLM
CAI	LLM	Multiple Tool Agents	-
AutoPentester	LLM + Modified Penetration Tree	LLM + RAG (articles) + Agents	LLM
CheckMate	Classical Planning+	LLM + Predefined Actions + Agents	LLM

symbolic grounding guarantees that valid actions are not overlooked, actions are only applied when their preconditions are satisfied, and all changes to the world state are explicitly and consistently tracked throughout the planning process. In addition, classical planning algorithms guarantee that if a valid sequence of actions exists, the planner must be able to discover it [38]. Moreover, every intermediate step in the resulting plan is logically consistent with the defined preconditions and effects, ensuring that each causal dependency is correctly maintained. Consequently, even for long action chains, the planner constructs solutions in a step-by-step manner, preserving the causal structure throughout the entire planning process.

In contrast, LLM-based planning relies on implicit, language-based reasoning [39], [40]. It lacks a persistent and structured memory of the world state, which makes it prone to forgetting past actions, repeating steps, or hallucinating outcomes, especially as the reasoning chain grows longer [41]–[43]. This also leads to skipped steps in the action sequence or invalid transitions. It also suffers from the limited context windows [44] (e.g., 8K–128K tokens), which restrict its ability to retain long-term planning structure [40], especially in complex tasks like penetration testing.

III. EVALUATION OF EXISTING PENTESTING SYSTEMS

A. Experimental Methodology & Setup

1) *Benchmark Datasets*: We adopt Vulhub [20], a community-maintained collection of containerized vulnerable environments, as the basis of our benchmark. From this repository, we randomly sampled 120 containers for evaluation. All target Docker images were anonymized, preventing the evaluation system from recognizing them as Vulhub challenges. Compared with recent work [5], [30], [45], our benchmark is the largest of its kind to date. We exclude puzzle-like challenges [46] such as those from HackTheBox [47], which emphasize more on CTF-style tricks. In addition, these challenge sets (e.g., [48]) have extensive public writeups, many of which are likely included in LLM training corpora, posing a risk of data contamination. To maintain experimental fairness and unbiasedness, we therefore do not incorporate them into our benchmark.

2) *Metrics*: To measure the actual penetration capability and progress of a pentest, we propose eleven milestones that cover the typical pentesting lifecycle. Please note that we did

not adopt the “sub-tasks” used in some prior work [30] as the metric. Because those sub-tasks emphasize completing specific activities rather than demonstrating meaningful impact. For example, a sub-task such as “web enumeration” can be checked off after enumeration was performed; that alone does not show that the tester discovered key information or used it effectively in later phases of the test. For this reason, we define milestones to assess real progress. For each pentesting engagement, we manually compare the testing process against the ground truth and judge which milestones have been achieved. The milestones are as follows:

- M1: Successfully enumerating network hosts, open ports, and running services.
- M2: Discovering multiple potential attack vectors (e.g., target services or software) without confirming the actual exploitable one.
- M3: Confirming and precisely localizing specific attack vectors susceptible to exploitation.
- M4: Obtaining or generating an exploitation command, code, or method.
- M5: Successfully executing the exploit that triggers the vulnerability or verifies the PoC.
- M6: Successfully executing arbitrary commands on the target system.
- M7: Establishing an interactive shell session with user-level privileges.
- M8: Discovering a viable privilege escalation method.
- M9: Establishing an interactive shell with elevated privileges (root on Linux/Unix, Administrator/SYSTEM on Windows).
- M10: Successful lateral movement.
- M11: Obtaining authentication credentials or private data in any format.

The milestones exhibit both sequential dependencies and parallel paths. In a typical linear progression, the sequential flow follows the pattern from M1 to M9, with each milestone requiring completion before advancing to the next. However, M10 and M11 can be pursued in parallel. For example, the tester may simultaneously explore lateral movement and privilege elevation on compromised systems, and user privacy and credentials can be obtained at any stage throughout the pentesting process. This milestone framework enables a more nuanced assessment of penetration progress and penetration capabilities.

3) *Baselines and Evaluation Criteria:* We chose four open-source, well-recognized pentesting systems, PENTESTGPT [30], PENTESTAGENT [45], CAI [4], and AutoPenTester [49], as the baselines. Related works such as AUTOATTACKER [33] and Penheal [32], as well as commercial systems like XBOW [50], do not release their code, making independent reproduction infeasible. CHAINREACTOR [26] provides an open-source implementation but narrowly focuses on privilege escalation attacks. Additionally, while several open-source pentesting toolkits [31], [35], [51] support automation primitives, they lack autonomous planning modules and thus cannot perform end-to-end automated pentesting. Lastly, traditional automated pentesting works are excluded because they neither offer reproducible implementations nor have automation capabilities.

Minimal Human Intervention. We noticed that some systems still rely on human intervention, such as interpreting system outputs, extracting key information, and guiding LLMs in choosing tools. We believe rigorous evaluation should minimize human involvement to ensure consistency, fairness, and unbiasedness. Accordingly, we followed the principle of minimal human intervention. For systems that could not operate in a fully hands-off-the-keyboard manner, we allowed only essential interactions, strictly limited to selecting default options, executing provided commands, reporting execution outcomes, etc, without offering external knowledge or guidance.

B. Comparative Evaluation of Existing Systems

In addition to the baselines representing the prior work on automated pentesting, we also evaluated the penetration capabilities of three out-of-the-box LLM agents: Claude Code [18] + Sonnet 4.5, Codex [52] + o4-mini, and Gemini Code Assistant [53] + Gemini Pro 2.5. We fed each system the same initial prompt with the task description and allowed them to use any tool that ships in a standard Kali Linux distribution. Beyond that, we provided no additional hints or human intervention. Any single step that stalled for more than two hours was terminated and counted as a failure. For PENTESTGPT [30], PENTESTAGENT [35], and CAI [4], we employed the most powerful LLM that each system supports. We measured the percentage of targets that each system advanced to each milestone.

The results shown in Figure 1 indicate that Claude Code + Sonnet 4.5 consistently outperforms all other systems across almost all milestones. The performance of PENTESTGPT drops sharply after M1, indicating its limited pentesting ability without human intervention. Although the remaining systems completed the early milestones involving basic reconnaissance and enumeration, their performance diverged significantly once the workflow demands deeper reasoning and planning, as well as exploit development. PENTESTAGENT outperforms CAI, Codex, and Gemini Code Assist due to its online exploit-search strategy, but it still fails to make progress beyond M4. In contrast, Claude Code maintains strong performance through M7 and still achieves some success in later stages, demonstrating a significantly better capability in multi-step

penetration tasks. Please note that M8 to M11 correspond to lateral movement, privilege escalation, and credential leakage. Because the Vulhub dataset simulates single-application vulnerabilities, it may not provide the attack paths necessary to reach these milestones. Overall, these results highlight two key findings. First, the out-of-the-box Claude Code + Sonnet 4.5 demonstrates substantially stronger capabilities for automated penetration testing than all prior systems evaluated in this domain. Second, this level of capability is not uniform across LLM-based code agents: Codex and Gemini Code Assist fail to progress beyond basic scanning and enumeration, whereas Claude Code consistently performs a larger number of successful follow-on actions after initial discovery. A detailed analysis of each system’s penetration process and the factors causing the performance gaps is presented in §III-C.

C. Discussion on Capabilities and Limitations

To investigate the reasons behind the strong performance of Claude Code + Sonnet 4.5, we conducted an ablation experiment with two modified configurations: (1) replacing Claude Code with an alternative agent, OpenInterpreter, while retaining Sonnet 4.5 as the backend LLM, and (2) retaining Claude Code as the agent but substituting Sonnet 4.5 with GPT-o4-mini. All other setups were the same. The results show that both alternative configurations had a substantial drop in performance and failed to achieve M3 on any task. The pentesting processes also revealed two major weaknesses. First, the agents occasionally failed to proceed independently and required human input to determine the next step. Second, they generated redundant steps, such as creating local files or performing unrelated checks before executing actions like port scanning. Overall, these results suggest that the strong pentesting capability derives from the combination of both Claude Code’s agentic control and Sonnet 4.5’s underlying model capabilities, and that removing either component leads to a significant loss in capability.

We then analyzed the detailed pentesting processes of all systems to answer two key questions: how does Claude Code outperform other systems, and where does it still fall short?

1) *Advantages of Claude Code:* PENTESTGPT relies on a human-in-the-loop workflow, while other systems incorporate command-execution capabilities, enabling a higher degree of automation. The LLM code agents are better at iteratively debugging and modifying commands based on the execution results compared to CAI. Although PENTESTAGENT excels in searching and leveraging online exploits, its performance lags in non-exploit tasks, such as enumeration and application probing. Among those LLM agents, Claude Code stands out with several strengths. First, Codex and Gemini Code Assist discover narrower attack surfaces and often select slower and less effective approaches (e.g., excessive password brute-forcing or path enumeration), while Claude Code can discover broader attack surfaces. Second, Codex and Gemini Code Assist demonstrate limited self-reflection and adjustment capabilities. They often fail to recognize when a chosen route is unproductive and may remain stalled on the same